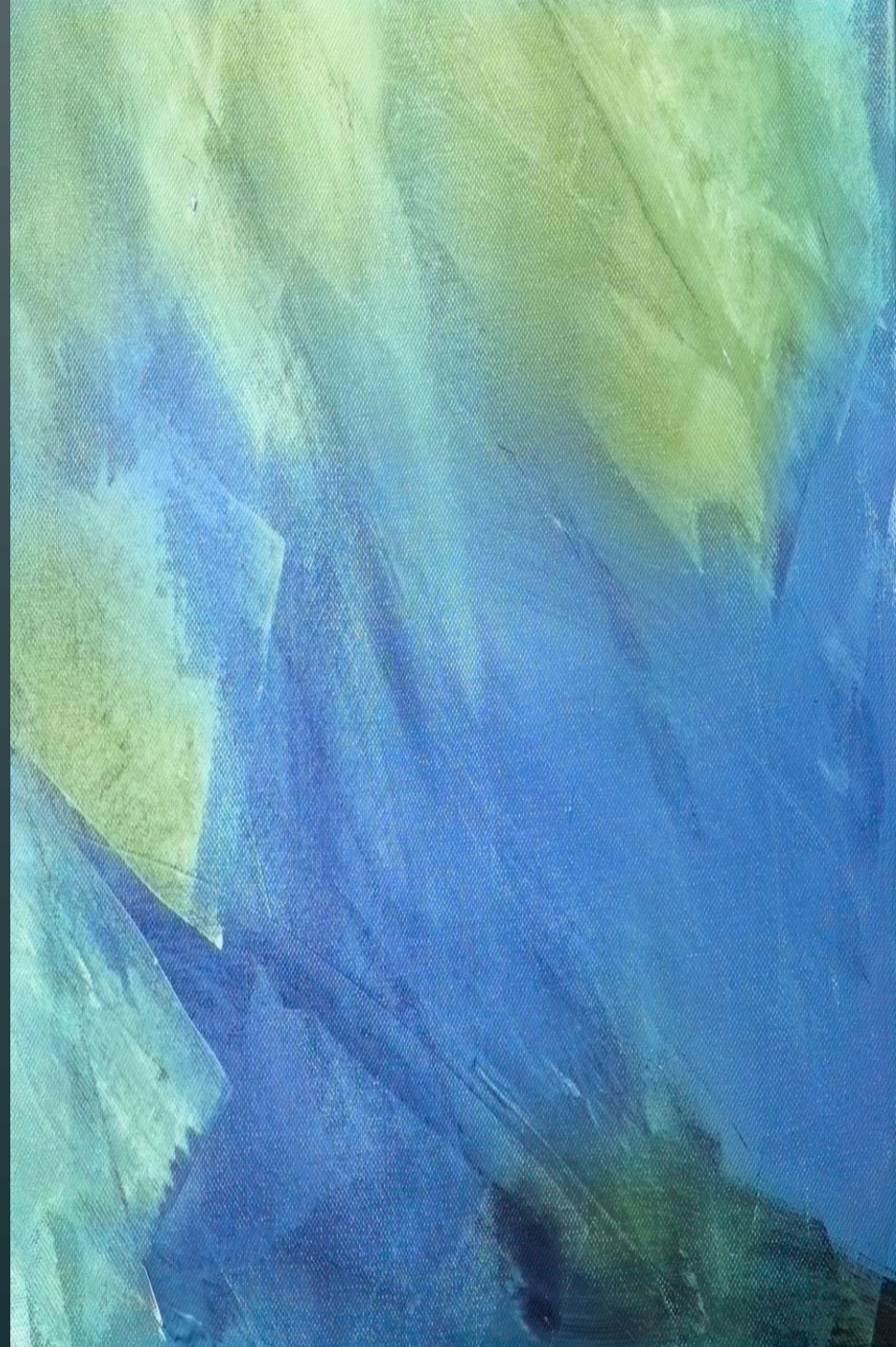
Interprocess Communication

RPI, service discovery and messaging

Wiwat Vatanawood
Duangdao Wichadakul
Nuengwong Tuaycharoen
Pittipol Kantavat



Patterns in this chapter

- Remote procedure invocation
- Circuit breaker
- Client-side discovery
- Self registration
- Server-side discovery
- Third-party registration
- Asynchronous messaging
- Message broker

Agenda

01 Overview of IPC

02 REST and gRPC

03 Handling partial failure

04 Service discovery

05 Asynchronous messaging

01

Overview of IPC

Technologies, interaction styles, APIs and message formats

Many IPC technologies



Synchronous

Request/response: REST or gRPC over HTTP



Asynchronous

Messaging: AMQP or STOMP



Text formats

Human-readable: JSON, XML



Binary formats

Compact: Avro, Protocol Buffers

Two dimensions of interaction style

How many services?

One-to-one: exactly one service

One-to-many: several services

Does the client wait?

Synchronous: expects a timely reply

Asynchronous: doesn't block

Interaction styles

	One-to-one	One-to-many
Synchronous	• Request/response (Need to wait for the response)	—
Asynchronous	• Asynchronous request/response • One-way notifications	• Publish/subscribe • Publish/async responses

Asynchronous interactions

A one-way notification may still receive a technical acknowledgment such as "message delivered." That is different from an application response containing a result.

nteraction style	Meaning	Example
Asynchronous request/response	A sender sends a request to one receiver, which sends a response later.	You request a report. The report service generates it and later sends you a "Report ready" message.
One-way notifications	A sender sends a message to one receiver, with no application-level reply expected.	An application sends a log entry to a logging service and continues its work.
Publish/subscribe	A sender publishes an event to a topic	receivers subscribe to that topic receives it.
Publish/async responses	A sender publishes a request to multiple receivers, which can each respond later, independently.	A travel application asks several airlines for prices. Each airline returns its quote when ready.

Asynchronous interactions

	Publish/subscribe	Publish/async responses
Purpose	Announce an event: "Something happened."	Ask multiple receivers: "Please provide a result."
Replies expected?	No	Yes, arriving independently later
Example	Announce an order to inventory, email, and shipping services	Ask several airlines for price quotes

Async does not mean "no response." It means the sender does not have to wait for that response before continuing.

Defining APIs

Hide the implementation

Expose useful functionality only

Change freely

The implementation can change without hurting clients

Use an IDL (Interface Definition Language)

Define the API precisely with an interface definition language

Today: OpenAPI for REST, Protocol Buffers for gRPC, AsyncAPI for messaging.

Message formats

Text

JSON, XML

Human-readable, self-describing

Binary

Protocol Buffers, Avro

Compact and efficient

02

REST and gRPC

Synchronous remote procedure invocation

Remote procedure invocation (RPI)

Request and response

The client sends a request; the service processes it and replies

RPI is a *request/response* pattern — a reply is always expected.

Blocking or reactive

Some clients block; others are reactive and non-blocking

client **reacts to the response when it arrives**, instead of keeping a thread blocked while waiting

Timely reply

The client assumes the response arrives quickly

Patterns map: communication style

Communication style

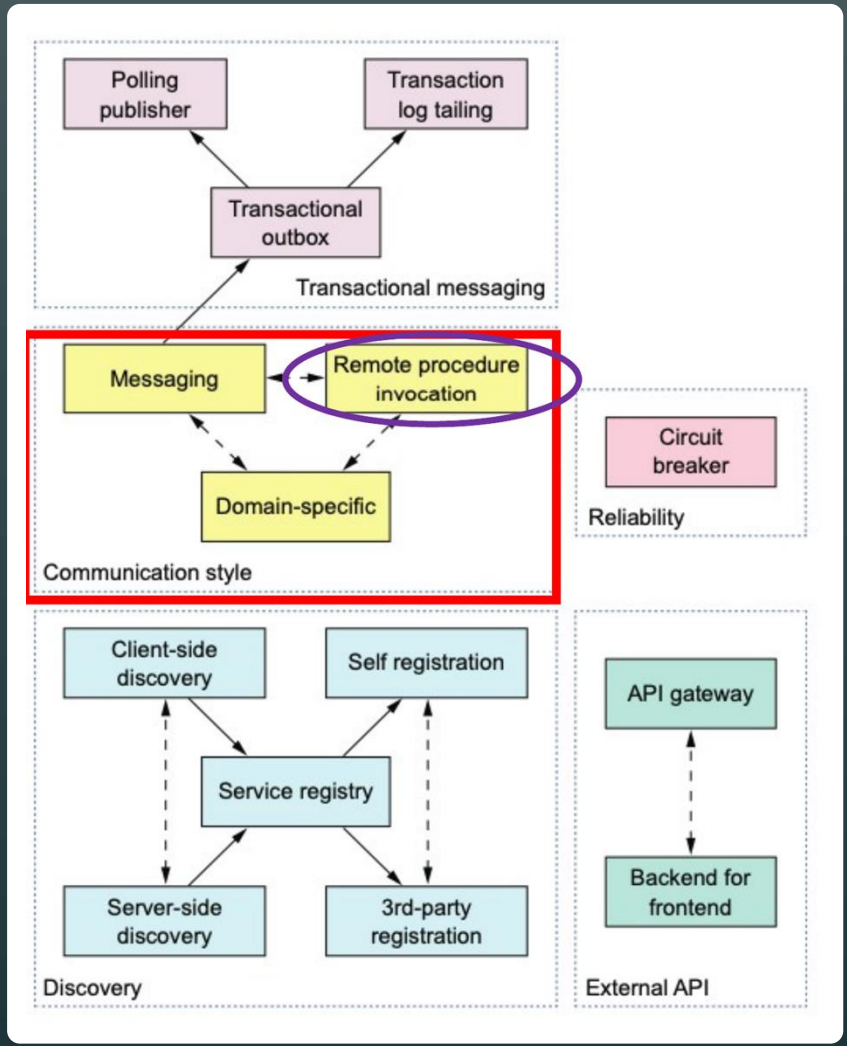
Remote procedure invocation or messaging

Reliability

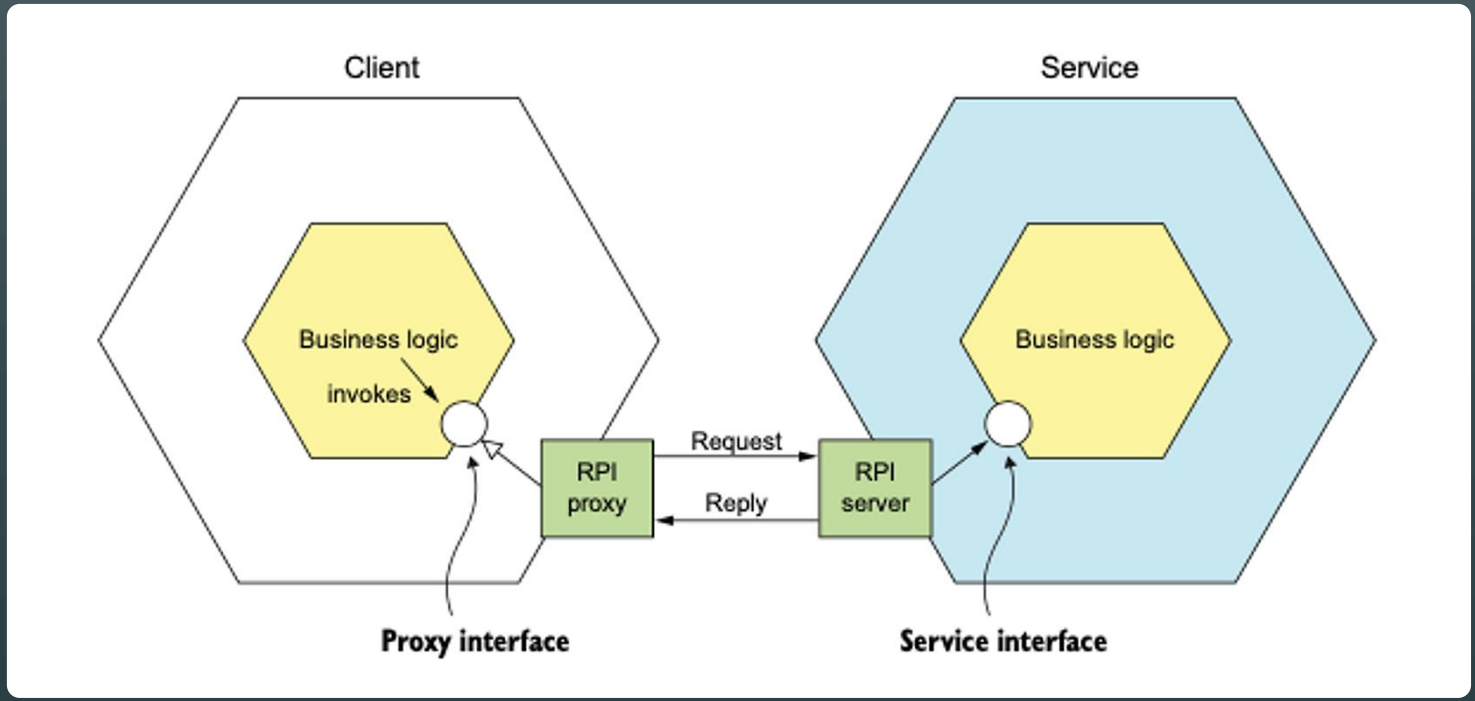
Circuit breaker

Discovery

Four patterns around a service registry



How RPI works



1. Business logic calls an RPI proxy interface
2. The RPI proxy sends the request
3. The RPI server adapter calls the business logic

REST

REST is an architectural style

Roy Fielding, who created REST, defines it as:

> *REST provides a set of architectural constraints that, when applied as a whole, emphasizes scalability of component interactions, generality of interfaces, independent deployment of components, and intermediary components to reduce interaction latency, enforce security, and encapsulate legacy systems.*

Source: Fielding's dissertation, ics.uci.edu/~fielding/pubs/dissertation

Web grows → HTTP designed → Fielding observes why it works well → Fielding formalizes those principles as REST

REST & HTTP

	HTTP	REST over HTTP
Protocol	✓	✓
Stateless requests	✓ (built in)	✓ (inherited)
Uniform resource URLs	not required	required
Correct verb semantics (GET for retrieval, POST for creation, etc.)	not required	required
Cacheable responses	supported but not required	required
Hypermedia links (HATEOAS)	not required	required (by Fielding)

Using REST

Resource

A business object, or a collection: Customer, Product

HTTP verbs + URLs

Manipulate resources referenced by URLs

Representation

GET returns the resource, often as JSON

Specifying REST APIs

Use an IDL

Define your APIs precisely

REST had none at first

Unlike CORBA and SOAP

OpenAPI Specification

The most popular REST IDL; evolved from Swagger

Today: OpenAPI 3.1 is current, and fully aligned with JSON Schema.

REST: trade-offs

Benefits

- Simple and familiar
- Easy to test: Postman, curl
- Request/response built in
- Firewall friendly
- No broker needed

Drawbacks

- Request/response only
- Both sides must be running
- Clients must find instances
- Many resources = many calls

gRPC



Cross-language

A framework for clients and servers in many languages



Protocol Buffers IDL

Google's language-neutral serialization



Request/response and streaming

Binary messages over HTTP/2

gRPC API for Order Service (excerpt)

```
service OrderService {
    rpc createOrder(CreateOrderRequest) returns (CreateOrderReply) {}
    rpc cancelOrder(CancelOrderRequest) returns (CancelOrderReply) {}
    rpc reviseOrder(ReviseOrderRequest) returns (ReviseOrderReply) {}
    ...
}
message CreateOrderRequest {
    int64 restaurantId = 1;
    int64 consumerId = 2;
    repeated LineItem lineItems = 3;
    ...
}
message LineItem {
    string menuItemId = 1;
    int32 quantity = 2;
}
message CreateOrderReply {
    int64 orderId = 1;
}
...
```

gRPC: trade-offs

Benefits

- Rich set of update operations
- Efficient, compact messages
- Bidirectional streaming
- Many languages

Drawbacks

- More work for JavaScript clients
- Older firewalls may block HTTP/2

Today: Browsers use gRPC-Web or Connect; HTTP/2 is now widely supported.

Backend for Frontend (BFF)

A BFF typically:

- **Combines data** from multiple services.
- **Shapes responses** for its frontend.
- **Reduces network calls**, especially useful on mobile connections.
- Handles frontend-specific concerns, such as session management.

The trade-off is an extra service to maintain. A BFF is most useful when different frontends have substantially different needs; a simple application may work well with one shared backend.

API gateway & BFF

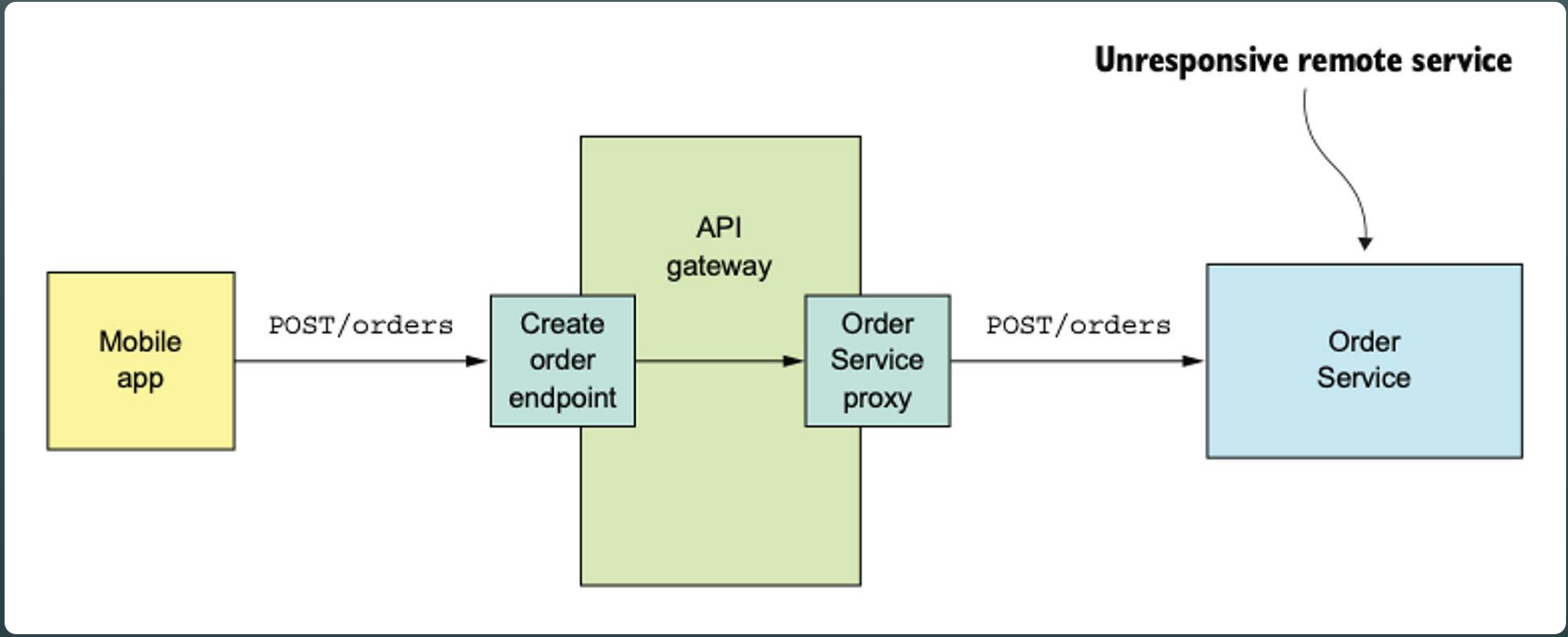
Aspect	API gateway	BFF
Main purpose	Provide a controlled entry point to services	Make backend data convenient for a particular frontend
Who it serves	Usually many types of clients	A particular frontend, such as a mobile app or website
Typical responsibilities	Routing, authentication checks, rate limits, request logging	Combining service results, selecting fields, formatting responses
Example	Route `/orders` to the order service and enforce request limits	Combine orders and delivery updates into one mobile screen response

03

Handling partial failure

The circuit breaker pattern

The risk of partial failure



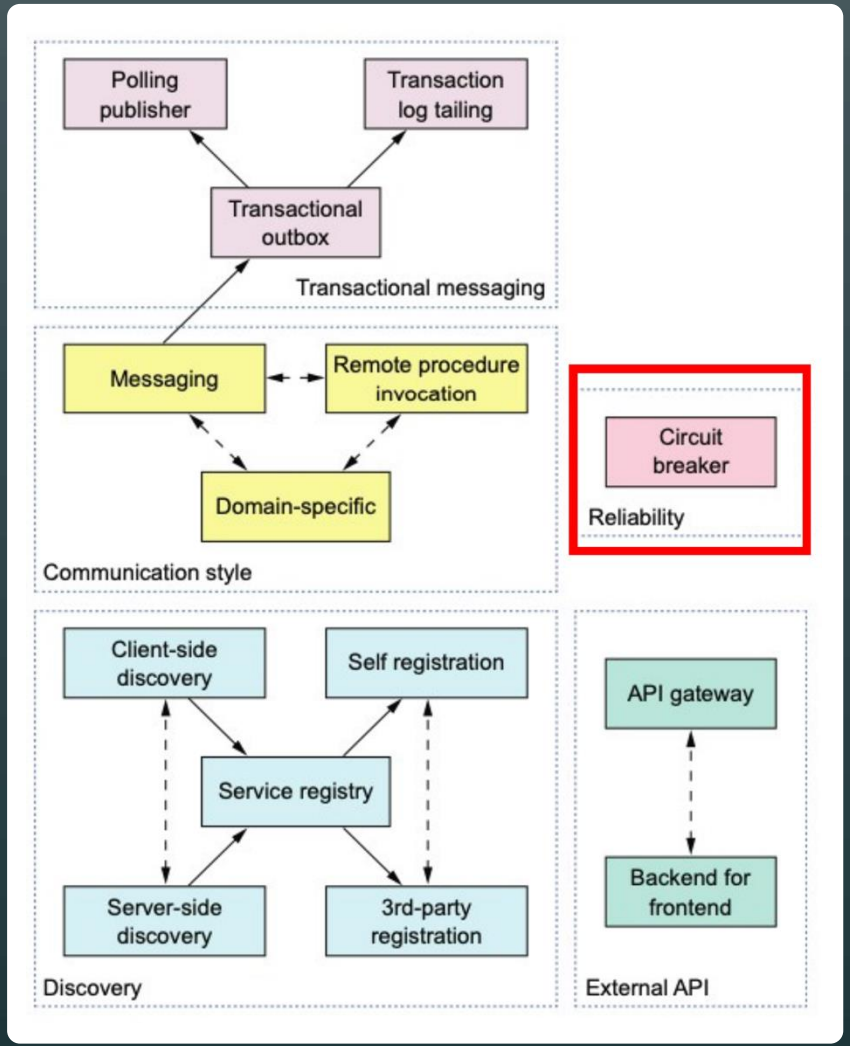
Synchronous calls
Risk partial failure

The API gateway
Must protect itself from unresponsive services

Patterns map: reliability

Circuit breaker

The reliability pattern for synchronous calls



Preventing partial failure



Part 1

Design RPI proxies to handle unresponsive services



Part 2

Decide how to recover from a failed service

Part 1: robust RPI proxies

Timeouts

Never block forever; always use a timeout

Limit requests

Cap outstanding requests to each service

Circuit breaker

Trip when the error rate passes a threshold; fail fast

Today: Use Resilience4j (Hystrix is in maintenance mode) or a service mesh.

Example tools for robust RPI proxies

Option	What it is	Where protection happens
Resilience4j	A Java library providing circuit breakers, concurrency limits, and timeout handling	Inside your application
Service mesh, such as Istio	Infrastructure that manages communication between services through proxies	In the network layer between services

Amazon: timeouts, retries and jitter

Timeouts
Pick an acceptable false-timeout rate, e.g. 0.1%, and set the timeout from it

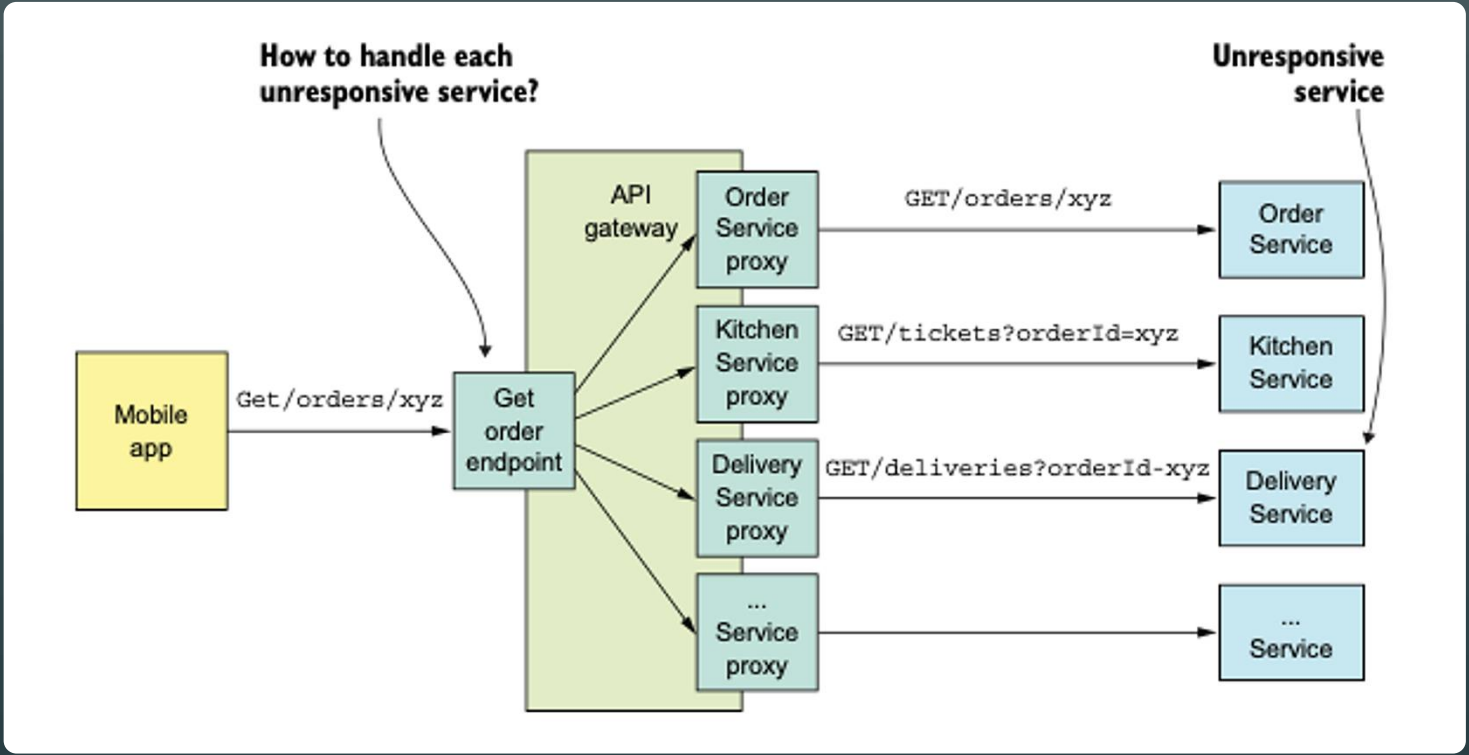
Capped exponential backoff
Wait longer between retries, up to a maximum

Jitter
Add randomness so clients don't all retry at the same moment

Read more: Amazon Builders' Library: Timeouts, retries and backoff with jitter

Lesson: Retry safely: backoff, jitter and idempotent APIs.

Part 2: recovering from failure



Option 1
Return an error to the client

Option 2
Return a fallback value

Plan per service
A strategy for each service you call

Example of Fallback values

Fallback type	Example
Previously saved data	Show the last known delivery location, labeled with its update time
Default value	Show popular products when personalized recommendations are unavailable
Partial result	Return order details without delivery information
Explicit “unknown” value	Return `deliveryStatus: "unknown"` so the app can display an appropriate message

04 Service discovery

Finding service instances in a dynamic environment

Why we need service discovery



Calling a service

Needs the IP address and port of an instance



Traditional apps

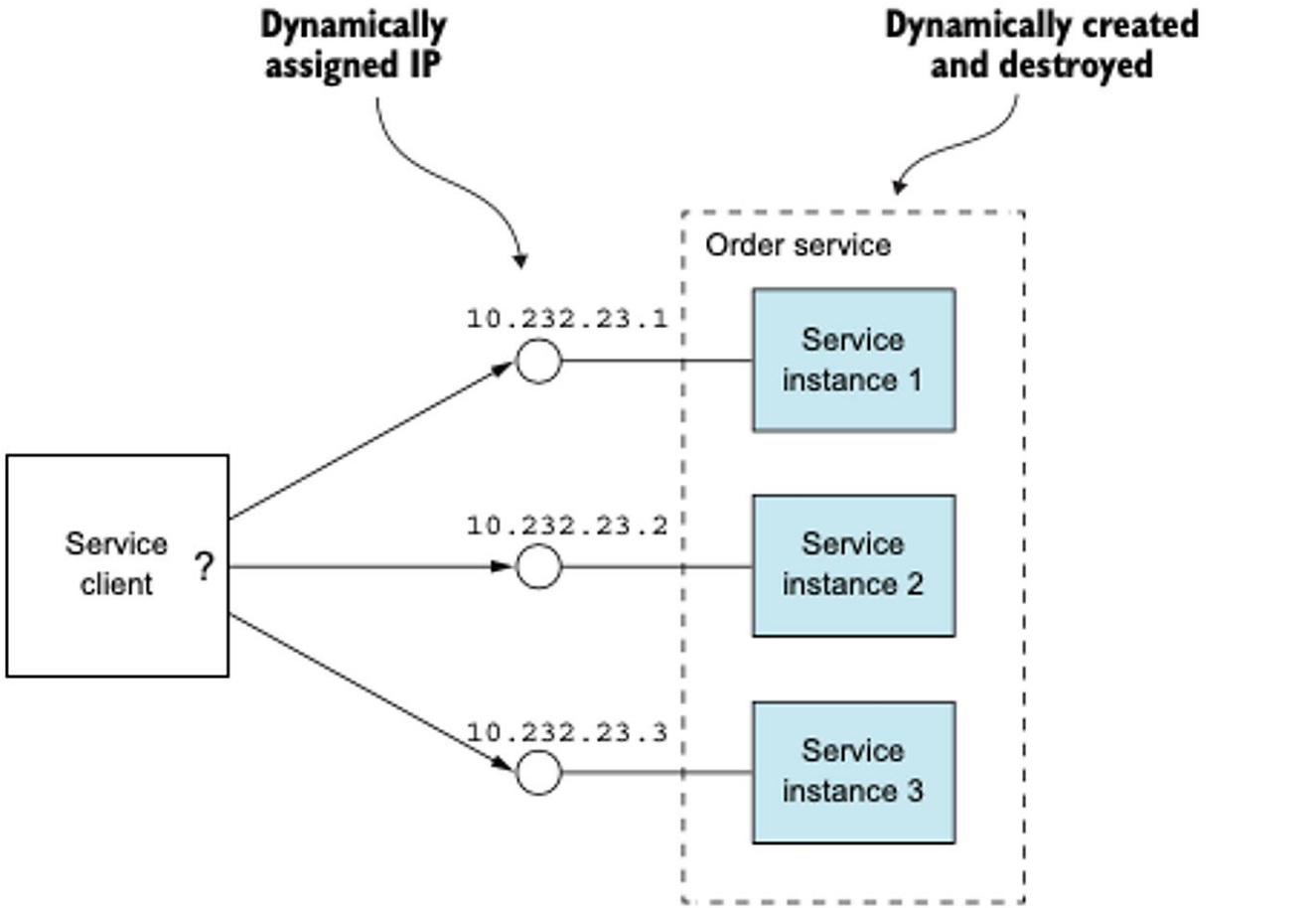
Network locations are usually static



Cloud microservices

Locations are dynamic and keep changing

Locations change constantly



Dynamic IPs

Instances get dynamically assigned addresses

Created and destroyed

Autoscaling, failures and upgrades

The service registry

The key component is a service registry: a database of instance locations.

Application-level

Services and clients interact with the registry directly

Platform-provided

The deployment infrastructure handles discovery

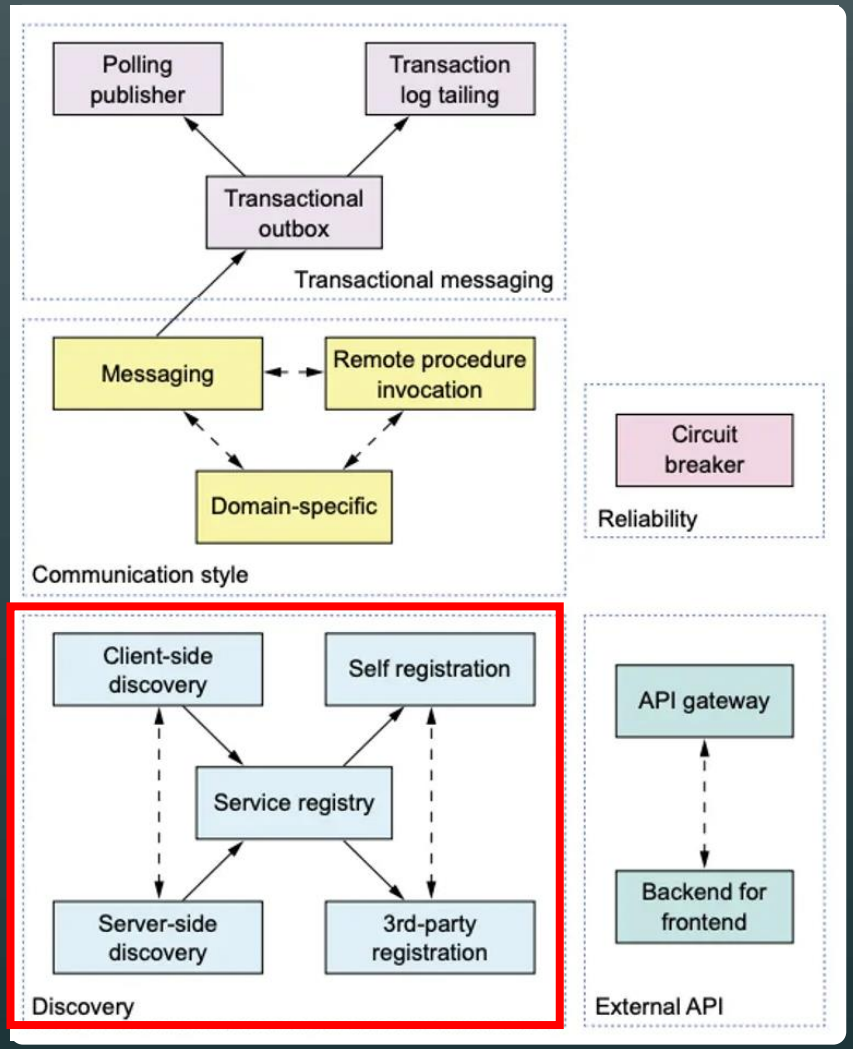
Patterns map: discovery

Application-level

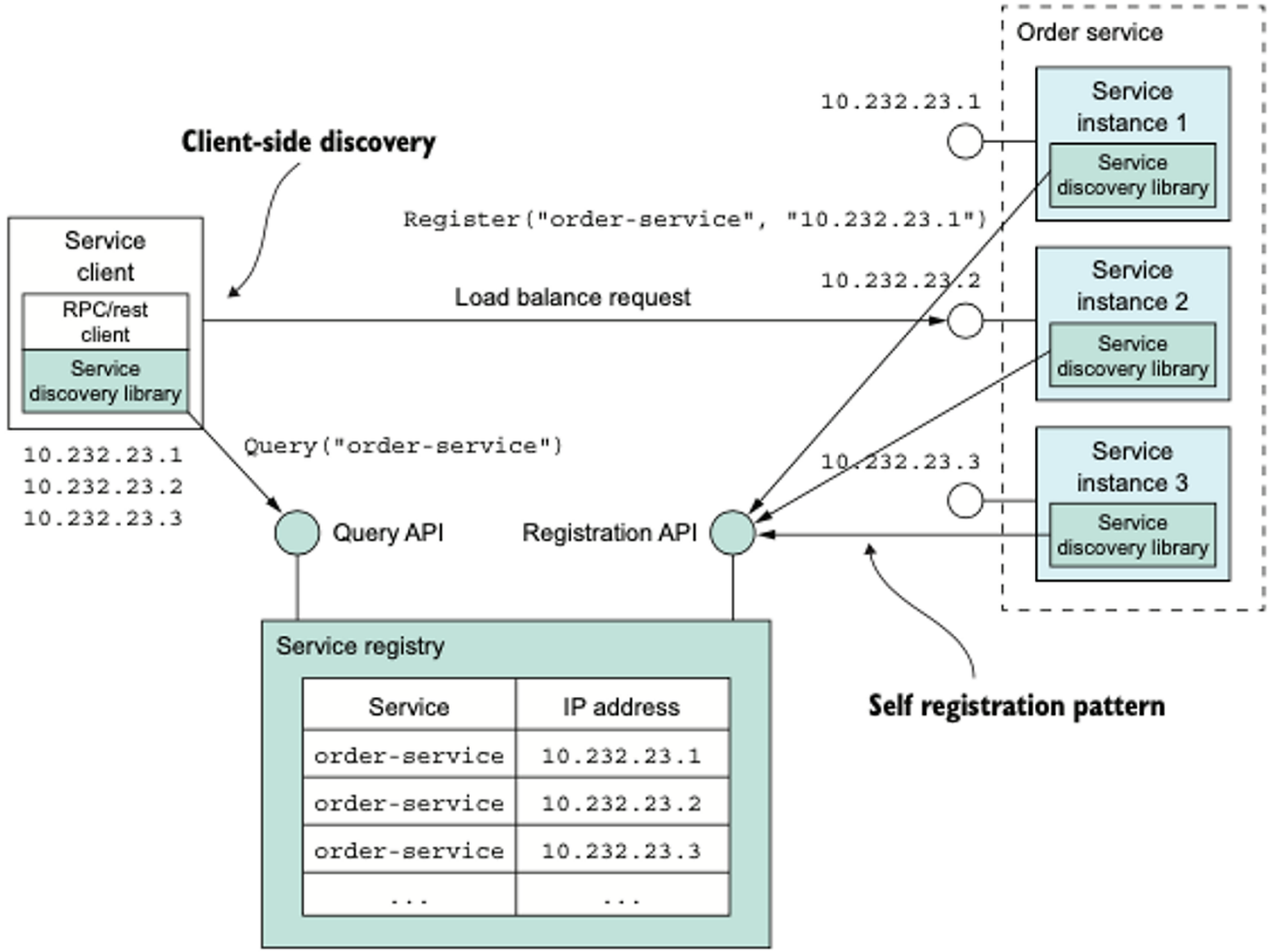
client-side discovery + self registration

Platform-provided

server-side discovery + 3rd-party registration



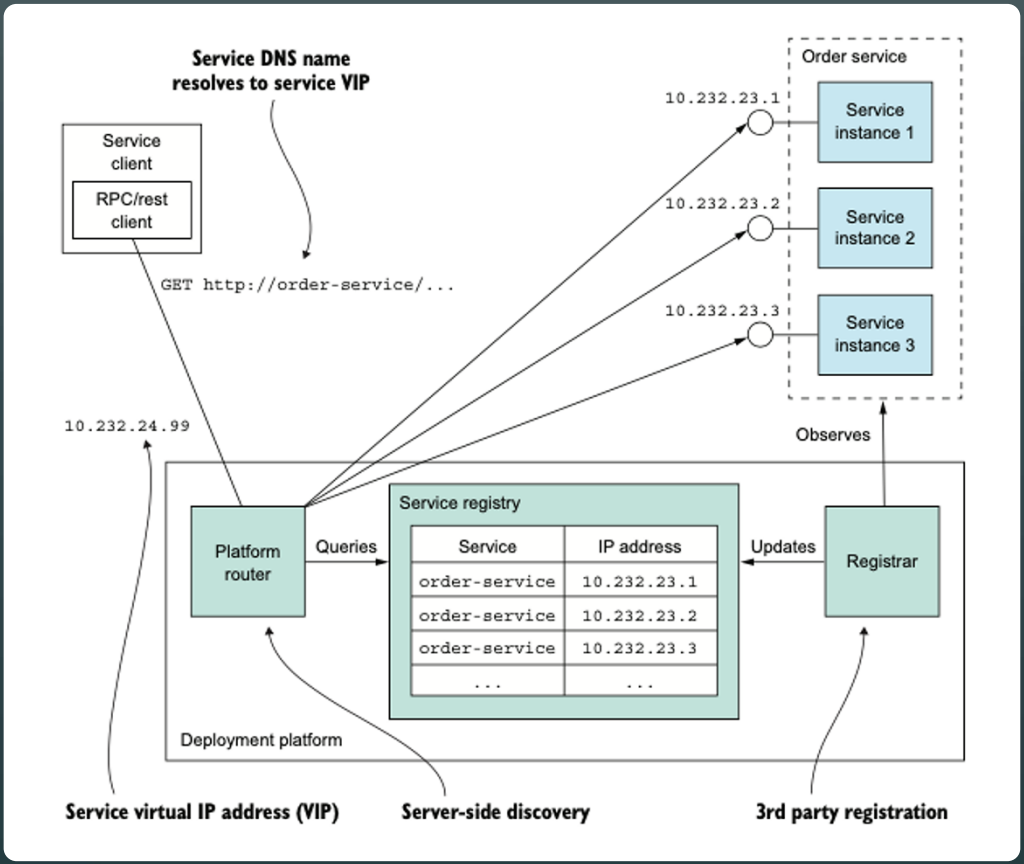
Option 1: application-level discovery



Client-side discovery
Clients query the registry and load-balance

Self registration
Instances register themselves with the registry

Option 2: platform-provided discovery



3rd-party registration

The platform registers instances

Server-side discovery

A platform router resolves a service DNS name to the service virtual IP address (VIP)

Today: Kubernetes Services and DNS do this for you.

Client side & Service side discovery

Approach	Pros	Cons
Client-side discovery	Client can customize load balancing and choose instances based on its needs. Can call instances directly without a separate routing hop.	Each client needs discovery and balancing logic, often through a library. Clients must handle stale instance lists and discovery failures. Supporting multiple programming languages adds maintenance.
Server-side discovery	Simpler clients: they use a stable address. Discovery and routing work across programming languages. The platform centrally tracks instances and manages routing.	Requires routing infrastructure to operate and troubleshoot. A separate proxy may add latency and become a bottleneck unless scaled. Clients have less direct control over instance selection.

05 Asynchronous messaging

Messages, channels and message brokers

Asynchronous messaging



Send a message

The client sends a request as a message



Don't block

The client doesn't wait for the reply

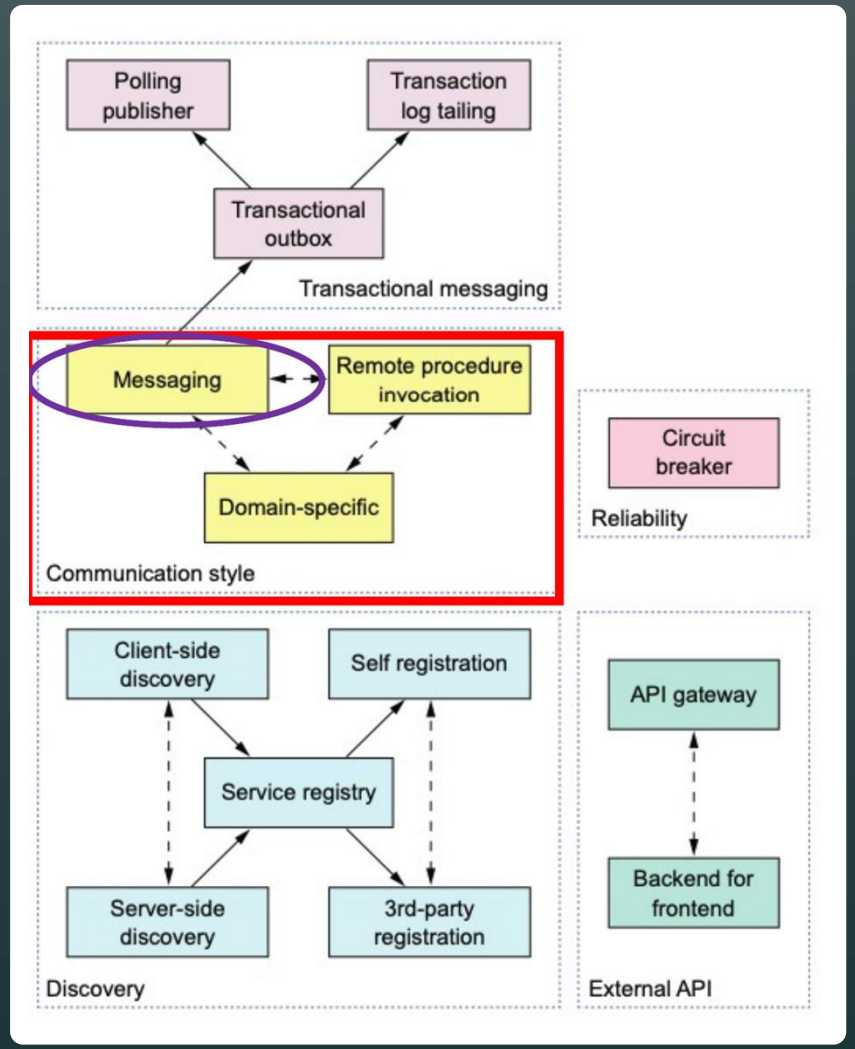


Two architectures

Brokerless or broker-based

Patterns map: messaging

Messaging
The asynchronous communication style



Messages and channels

Senders write messages to channels; receivers read them.



Document

Contains only data



Command

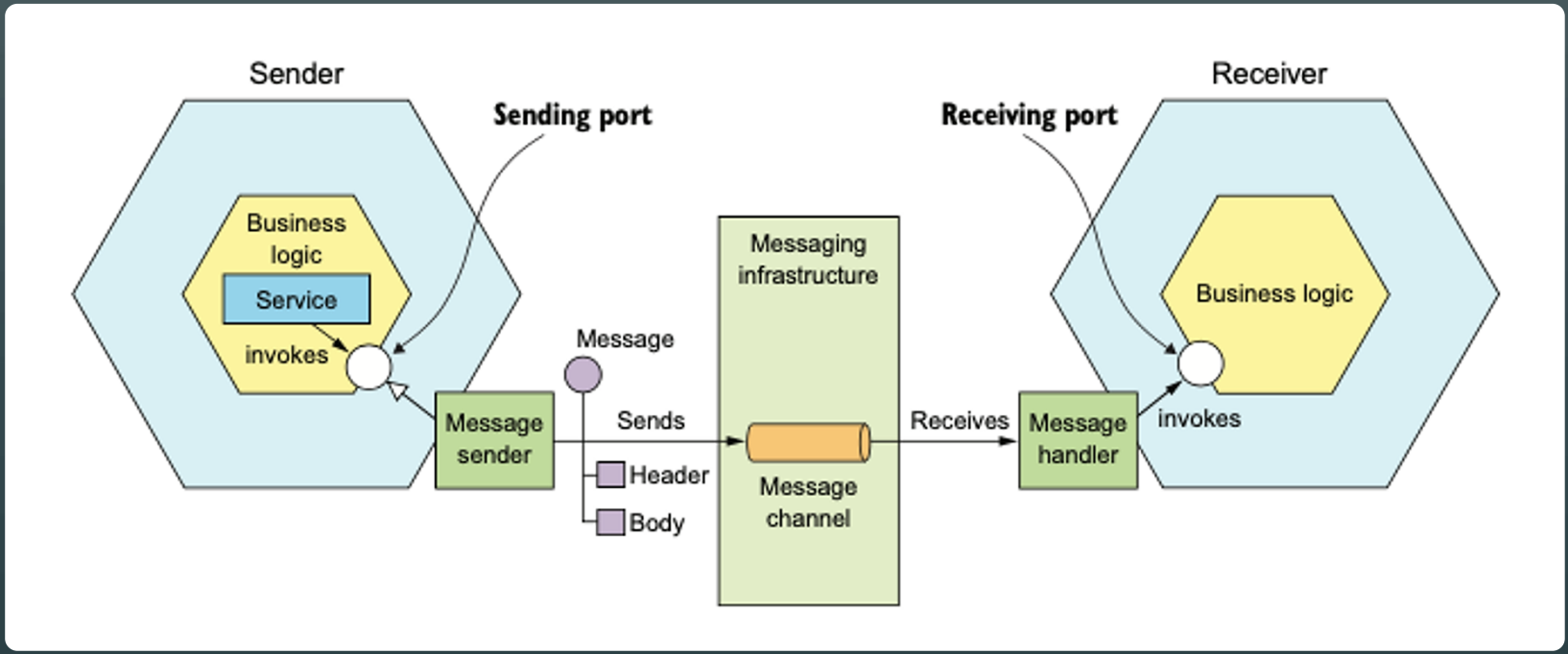
Like an RPC request



Event

Something notable happened

Messaging architecture



1. Sender's logic calls a sending port of the Message sender
2. The message travels on a channel
3. A Message handler calls the receiver's logic

Two kinds of channels

Point-to-point

Delivers to exactly one consumer

e.g. command messages

Publish-subscribe

Delivers to all attached consumers

e.g. event messages

Interaction styles with messaging

Async request/response

Asynchronous by nature, the receiver responds later

One-way notification

The sender sends message to a service, no wait; the service consumes the message; no reply

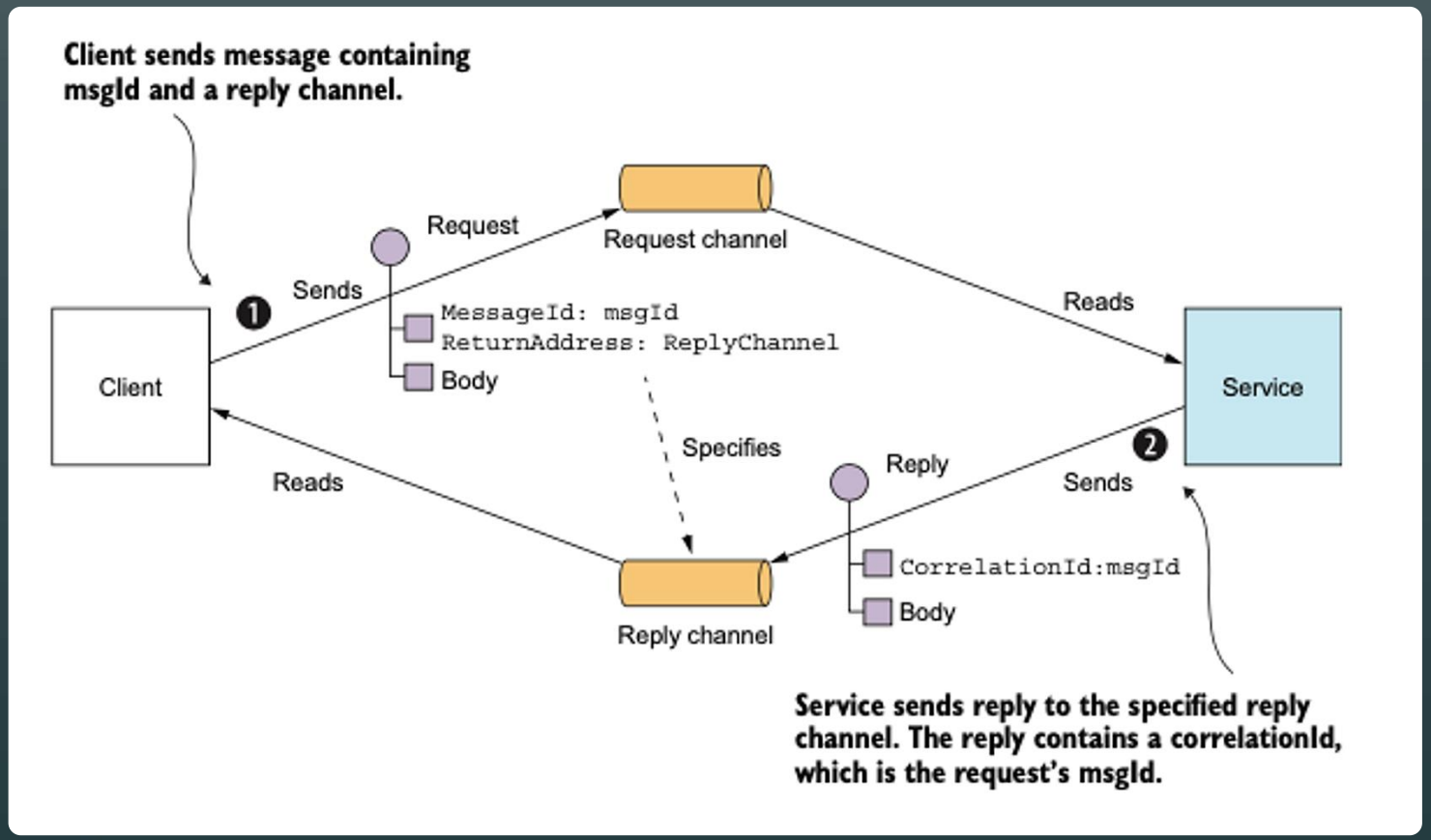
Publish/subscribe

The sender publishes a message to a pub-sub channel, many consumers, no reply

Publish/async responses A

The sender publishes a request to multiple receivers, which can each respond later, independently.

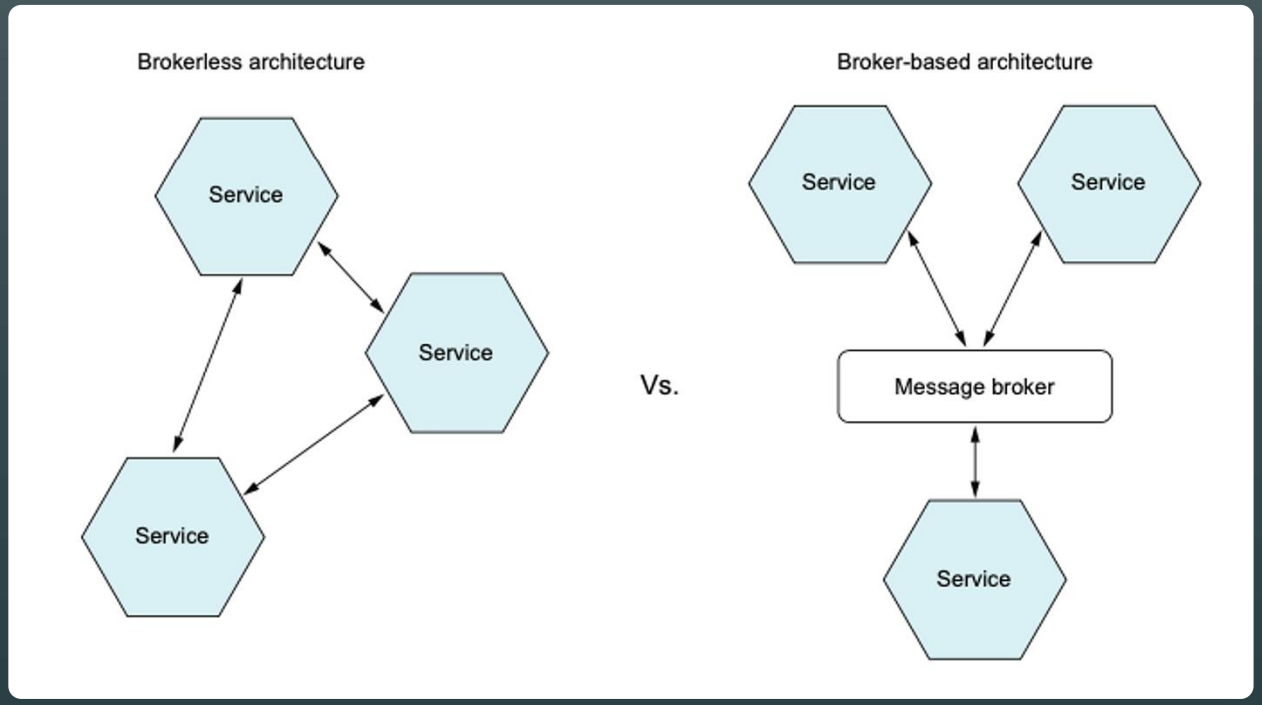
Asynchronous request/response



Request
Includes a reply channel and a message ID

Reply
Sent to the reply channel, carrying a correlation ID

Brokerless vs broker-based



Brokerless
Services exchange messages directly

Broker-based
All messages flow through a broker

Our focus
Broker-based messaging

Brokerless messaging: trade-offs

Benefits

- Less traffic, lower latency
- No broker bottleneck or single point of failure

Drawbacks

- Needs service discovery
- Both sides must be available

Broker-based messaging

A message broker is an intermediary that all messages flow through.

Location-free

The sender needn't know where consumers are

Buffering

Messages wait until consumers can process them

Brokers: ActiveMQ, RabbitMQ, Kafka · cloud: Amazon SQS/Kinesis, Google Cloud Pub/Sub

Broker-based messaging: trade-offs

Benefits

- Loose coupling: no discovery
- Message buffering
- Supports several interaction styles
- Explicit IPC

code makes communication between separate processes obvious

Drawbacks

- Possible bottleneck
- Possible single point of failure
- More operations work

Today: Kafka 4.0 (2025) runs without ZooKeeper, which simplifies operations.

Competing receivers and ordering



Competing receivers

Three instances read the same point-to-point channel



Out of order

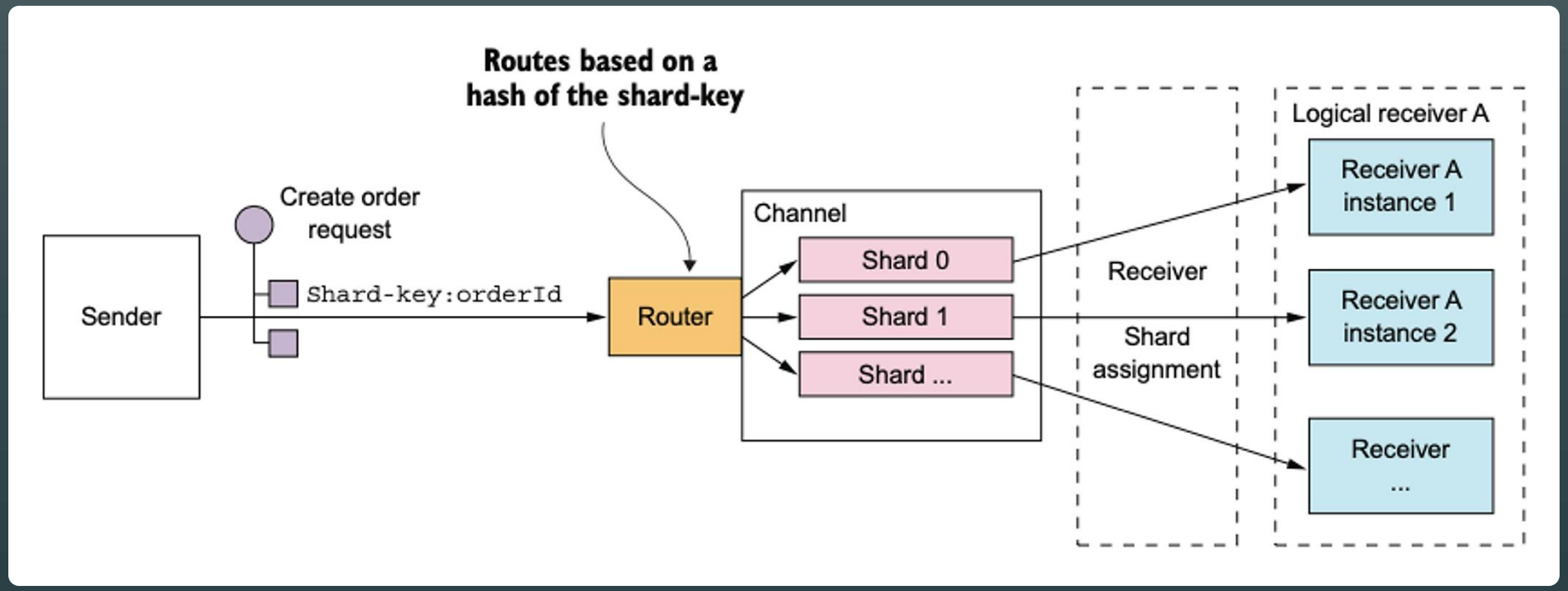
Created, Updated, Cancelled may be processed in the wrong order



Solution

Sharded (partitioned) channels: Kafka, Kinesis, RabbitMQ Stream (Super Streams)

Sharded channels preserve order



Shard key
Each Order event uses orderId

Same shard
All events for one order go to one shard

One consumer
Each shard is read by a single instance

Handling duplicate messages



Duplicates happen

Client, network or broker failures can redeliver a message



Example

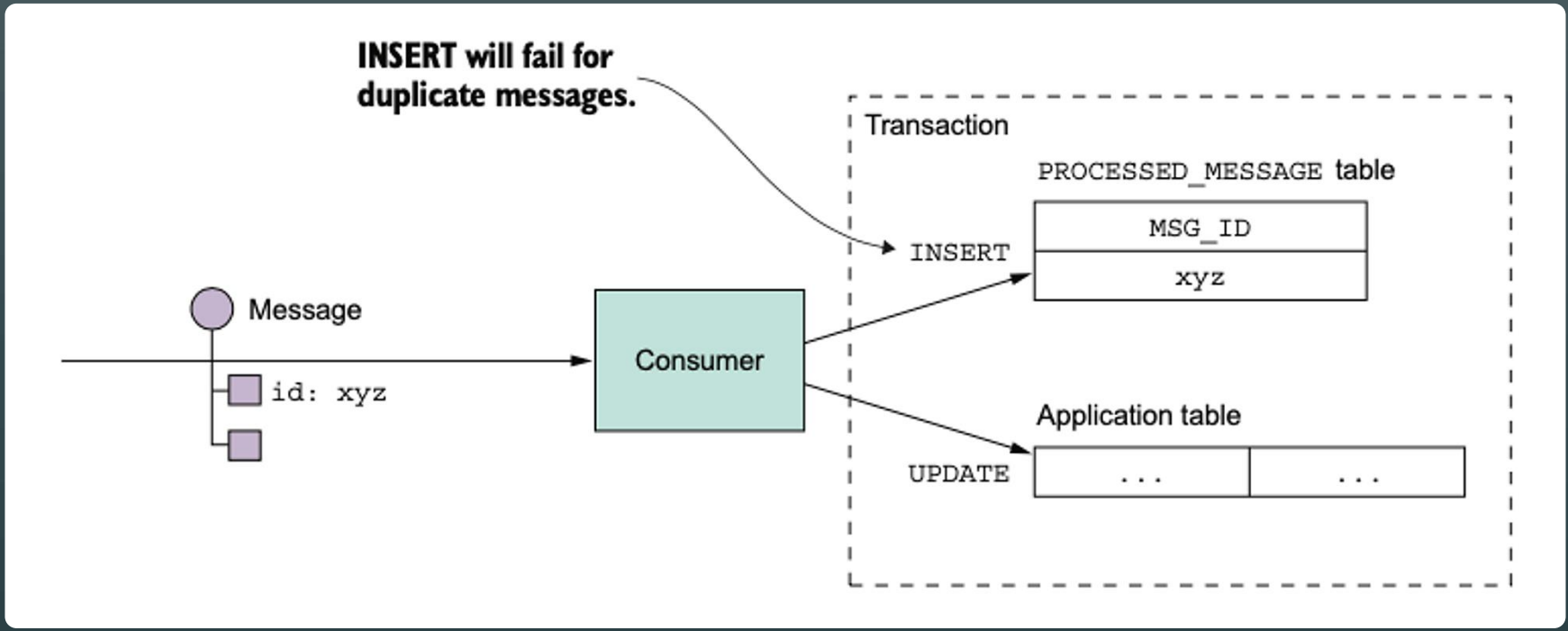
Authorizing a credit card must not happen twice



Solution

Discard duplicates by tracking message IDs

Tracking processed messages



Record IDs

Store processed message IDs in a table

Duplicate?

The INSERT fails, so the message is skipped

Kafka: messaging built at LinkedIn

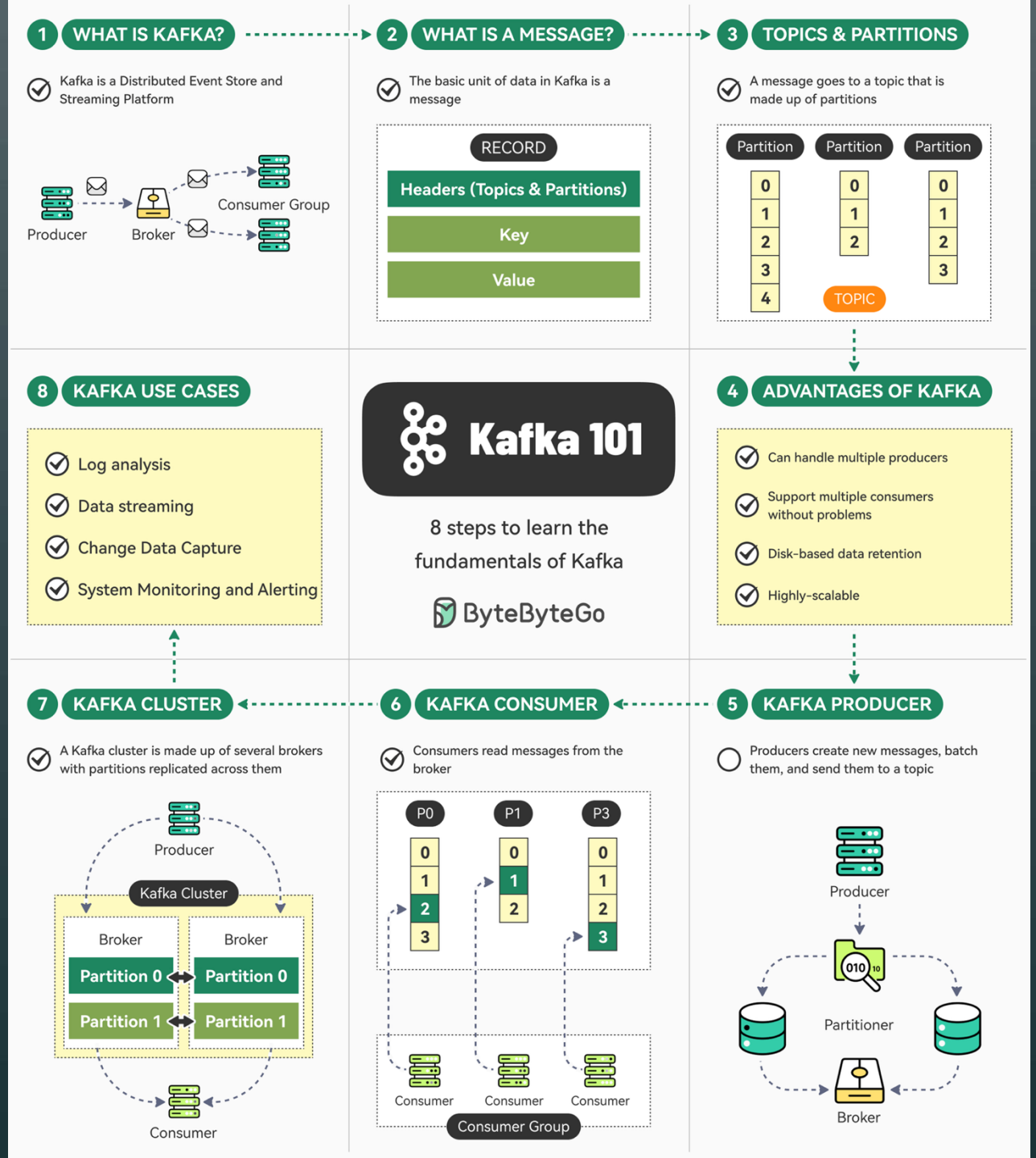
Built for volume
Collects and delivers high volumes of log data with low latency

Partitions
Each topic partition is a log: the sharded channels you just saw

At-least-once delivery
So consumers must be ready for duplicate messages

Read more: Kafka paper (Kreps et al., NetDB 2011)

Lesson: Partitions keep order; at-least-once means consumers dedupe.



Concept	What it is
Broker	A server in the cluster
Topic	A logical category of messages
Partition	The physical unit of storage and distribution
Broker's role	Hosts partitions from potentially many topics

Each partition also has replicas across brokers for fault tolerance:

Partition 0:
Broker 1 → Leader (handles reads/writes)
Broker 2 → Follower replica
Broker 3 → Follower replica

https://assets.bytebytego.com/diagrams/0246-kafka-101-8-steps-to-learn-the-fundamentals-of-kafka.png

What Brokers in Kafka do

1. Store messages

The most fundamental job. Each broker stores its assigned partitions as **append-only log files** on disk.

(Messages are never deleted immediately — they are retained for a configured period_

2. Accept writes from producers

When a producer sends a message, it connects to the broker that is the **leader** for that partition and writes to it. The broker acknowledges the write once it is stored.

3. Serve reads to consumers

A consumer connects to the broker hosting the partition and fetches messages starting from a given **offset**.

The broker does not push messages — consumers **pull** from brokers.

4. Manage replication

Each partition has one **leader broker** and one or more **follower brokers**.

The leader handles all reads and writes / follower(s) just continuously replicate from the leader

What Brokers in Kafka do

5. Handle consumer group coordination

Consumer Group: order-processors
- Instance 1 → reads Partition 0
- Instance 2 → reads Partition 1
- Instance 3 → reads Partition 2

When an instance joins or leaves, the broker triggers a **rebalance** — reassigning partitions among the remaining instances.

6. Cluster coordination (KRaft mode)

Brokers handle broker metadata such as which brokers are alive, who is the leader for each partition, etc. using KRaft protocol

RabbitMQ

RabbitMQ is a **message broker** built around the **AMQP protocol** (Advanced Message Queuing Protocol). It was originally developed by **Rabbit Technologies** in 2007 and is now maintained by **Broadcom** (previously VMware, previously Pivotal).

Core architecture differences

Kafka: Producer → Broker (partition/log) → Consumer pulls
RabbitMQ: Producer → Exchange → Queue → Broker pushes to Consumer

The hierarchy in RabbitMQ

RabbitMQ Node (server)
└── Virtual Hosts (vhosts) — isolated environments
    └── Exchanges — receive messages from producers
        └── Bindings — rules connecting exchanges to queues
            └── Queues — store messages for consumers

What Brokers in RabbitMQ do

1. **Accept messages from producers**
   Producer → connects to broker → publishes to exchange with routing key

2. **Route messages to queues**
   Broker → applies binding rules → copies message to matching queues

3. **Store messages in queues**
   Messages wait in queues until consumers are ready. The broker manages queue storage in memory and/or disk depending on configuration.

4. **Push messages to consumers**
   Broker → pushes message → Consumer
   Consumer → sends acknowledgment → Broker removes message

5. **Handle acknowledgments**
   ack — consumer processed successfully, broker removes message
   nack — consumer failed, broker can requeue or discard
   reject — similar to nack

What Brokers in RabbitMQ do

6. Cluster coordination

In a RabbitMQ cluster, nodes share metadata about exchanges, bindings, and queues. Queues can be:

- **Classic queues** — live on one node, replicated optionally
- **Quorum queues** — replicated across multiple nodes using Raft consensus (recommended for reliability)
- **Streams** — append-only log similar to Kafka partitions (added in RabbitMQ 3.9)

7. Dead letter handling

When a message cannot be processed (expired, rejected, queue full), the broker can route it to a **dead letter exchange** for separate handling.

Failed message → Dead Letter Exchange → Dead Letter Queue → inspection/retry

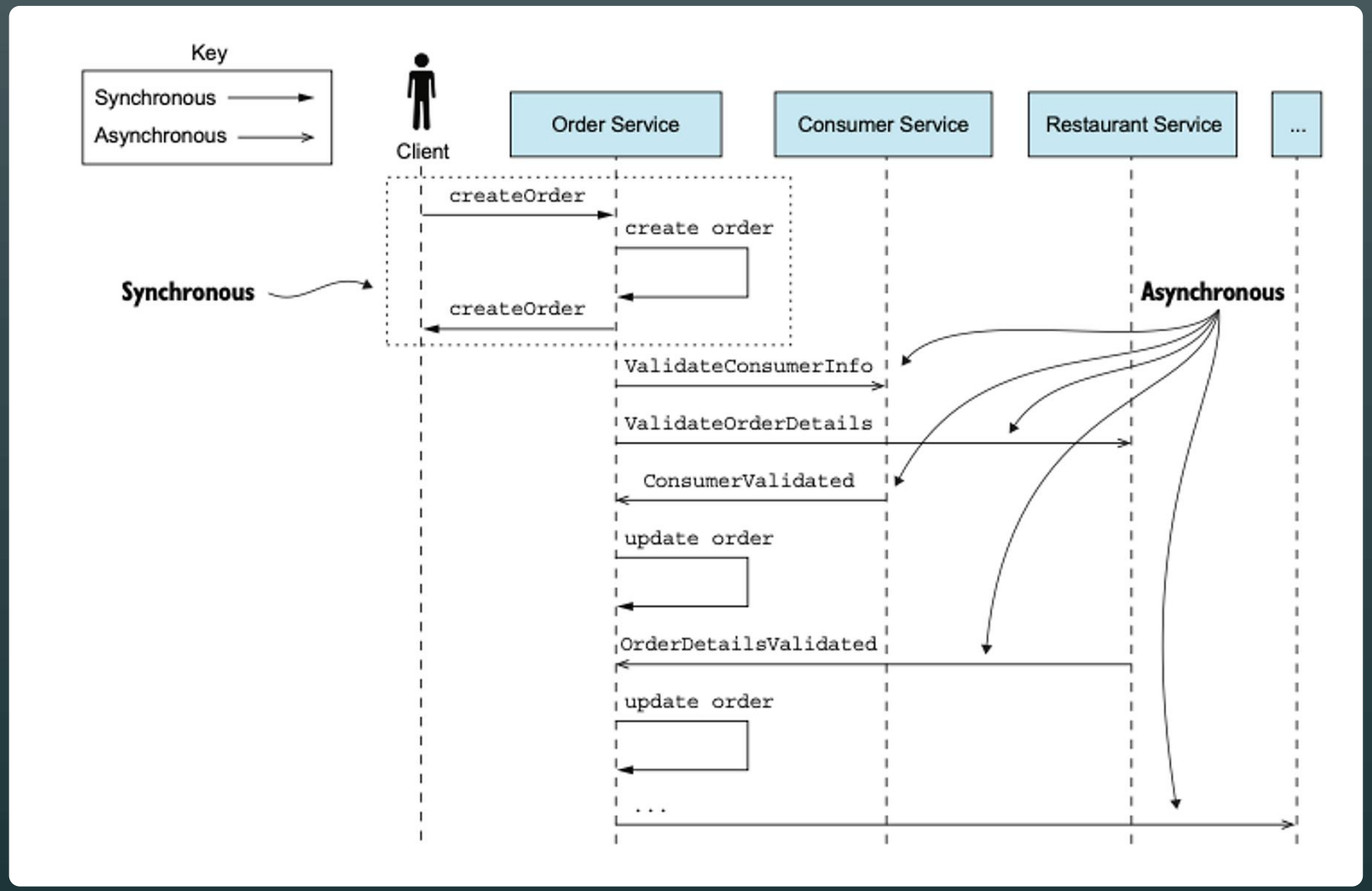
RabbitMQ vs Kafka key differences

Aspect	RabbitMQ	Kafka
Primary model	Message broker (push)	Distributed log (pull)
Message retention	Deleted after consumption	Retained for configured period
Replay messages	Not by default	Yes — consumers can re-read
Routing	Flexible — exchanges and bindings	Simple — topic and partition
Ordering	Per queue	Per partition
Throughput	High but lower than Kafka	Very high
Best for	Complex routing, task queues	High volume streaming, event log
Consumer model	Push to consumer	Consumer pulls
Protocol	AMQP (also STOMP, MQTT)	Custom binary protocol

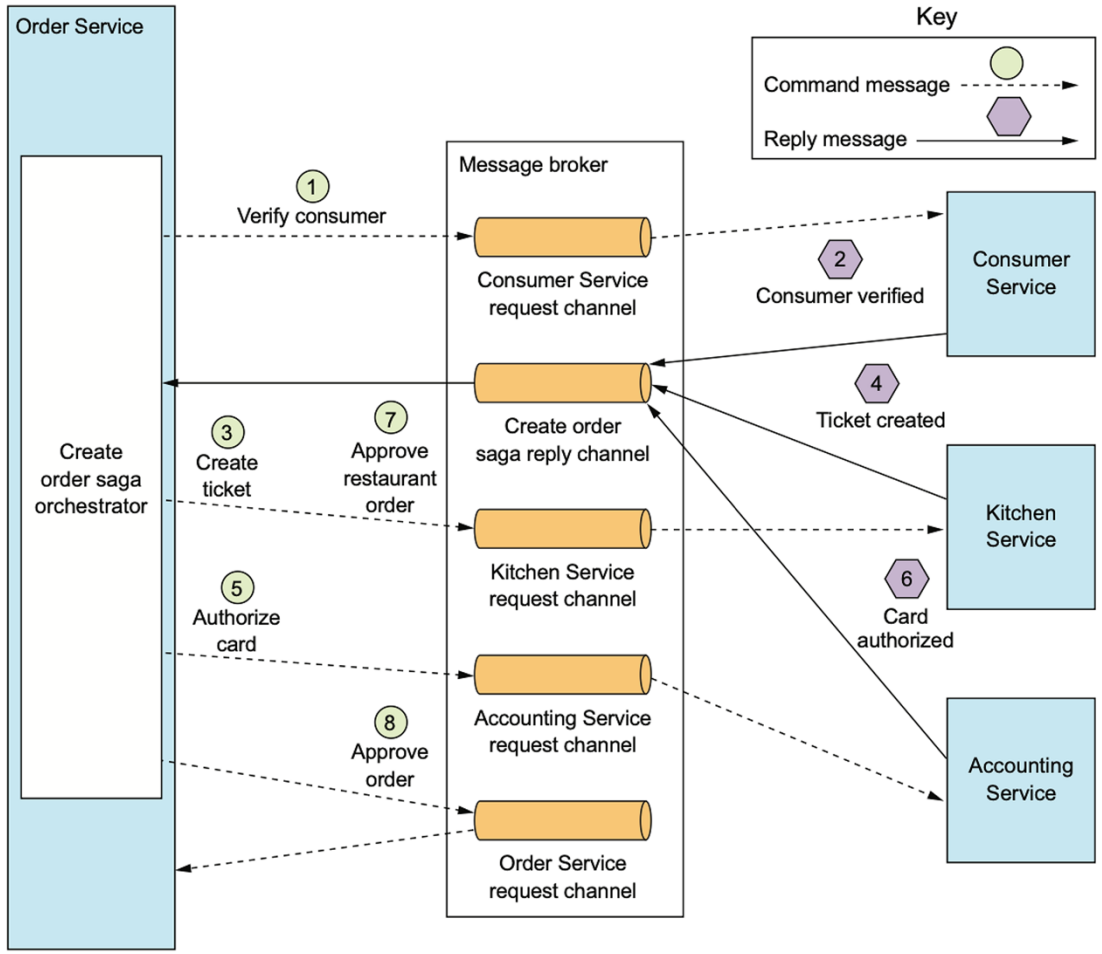
When to use RabbitMQ vs Kafka

Use RabbitMQ when	Use Kafka when
You need complex message routing	You need very high throughput
Messages should be deleted after processing	You need message replay
You need per-message acknowledgement	You need event streaming
Task queues (one worker processes each task)	Event sourcing
Lower volume, lower latency needed	Multiple consumers reading same data

Sync vs async interactions



Messaging in a saga: Create Order



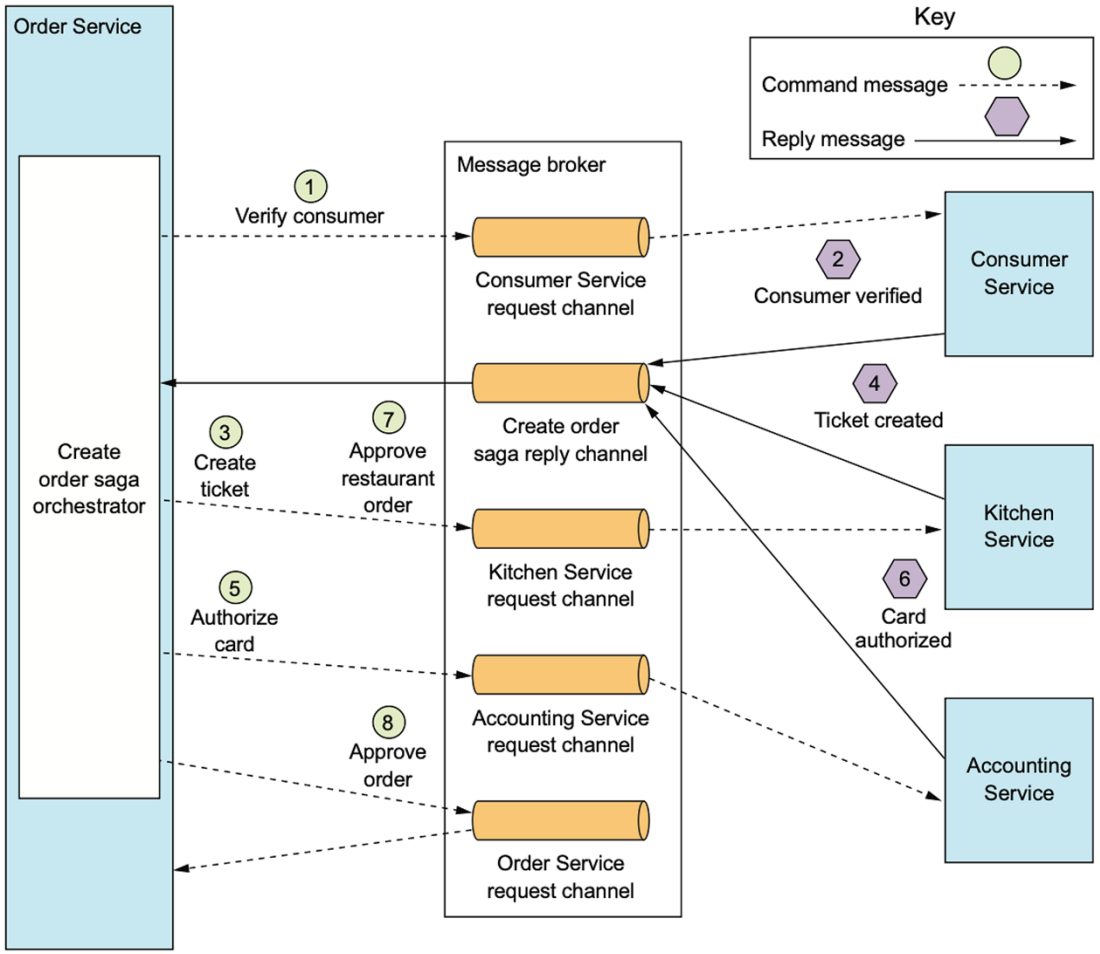
Saga is a design pattern for managing a business transaction across multiple services.

Saga uses a Saga orchestrator to manage a **sequence of smaller, local transactions**.

For example, the business operation is **"Create Order"** includes:

1. Verify the customer — Consumer Service.
2. Create a kitchen ticket — Kitchen Service.
3. Authorize the payment card — Accounting Service.
4 Approve the kitchen order and the overall order.

Messaging in a saga: Create Order



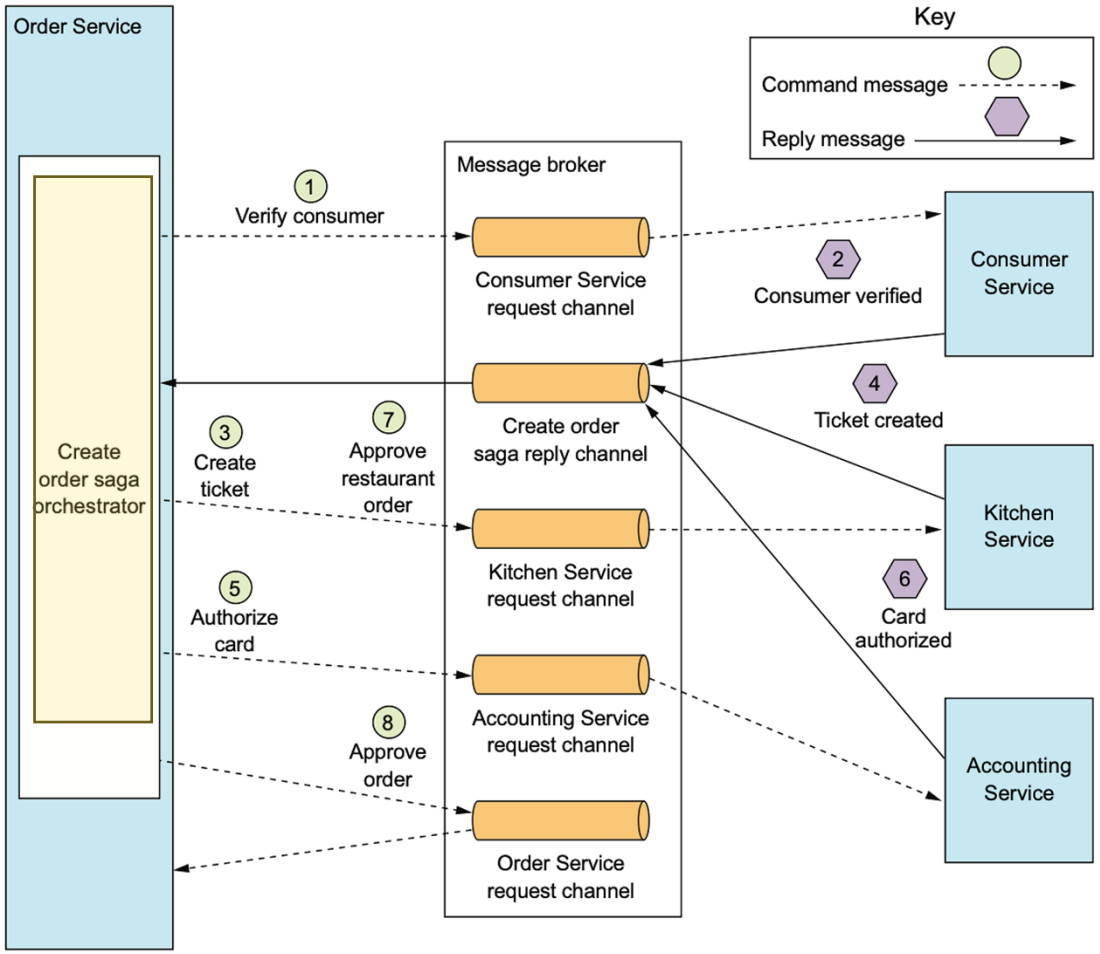
Verify customer → Success (committed)
Create kitchen ticket → Success (committed)
Authorize payment → Failed
↓
Compensate step 2: Cancel kitchen ticket
Compensate step 1: Reject order

Compensation is not a normal database rollback. Earlier steps have already committed.

Cancelling the ticket is a *new operation* that counteracts creating it. Compensation can also fail, so it may need retries.

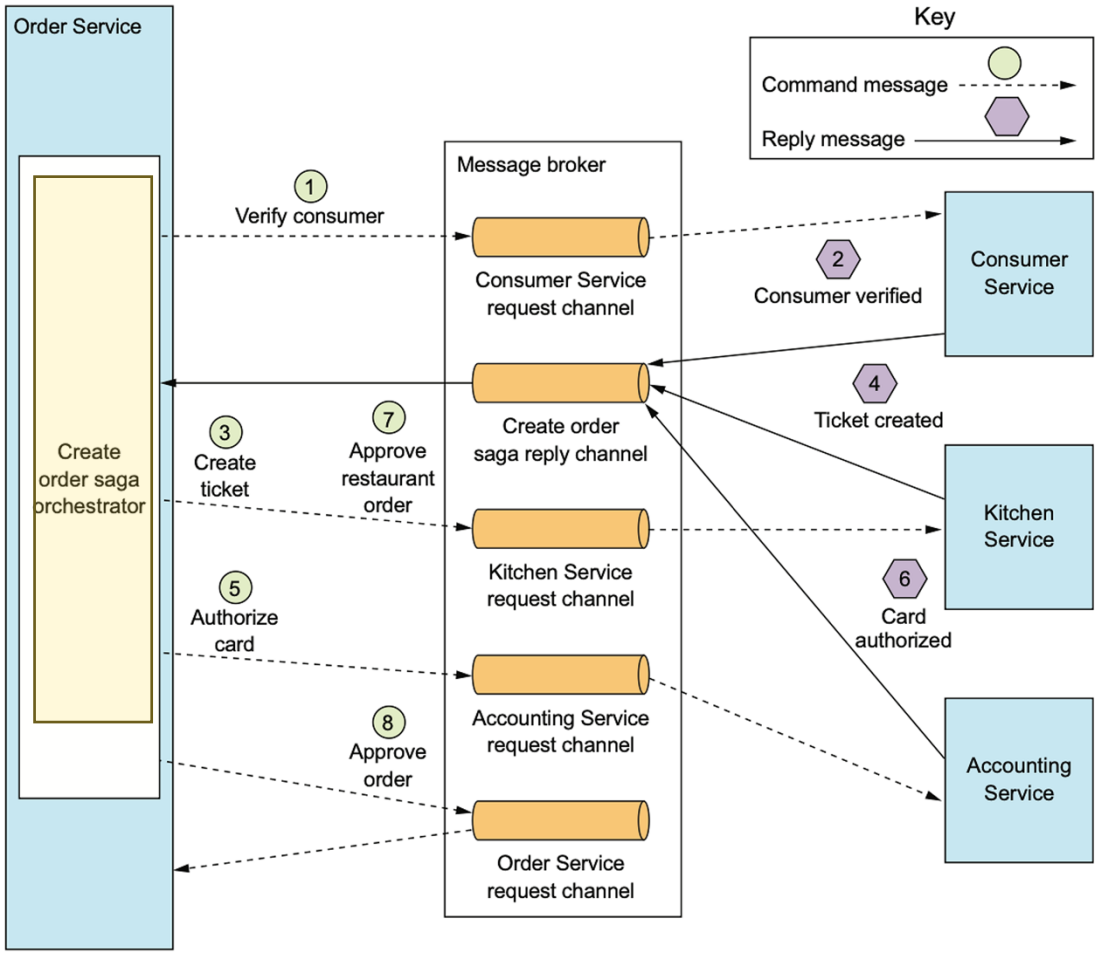
n a saga, each step is a **separate local transaction** that has already committed:

Messaging in a saga: styles



Style	How steps are coordinated
Orchestration — จากรูปด้านซ้ายเป็นแบบนี้	A coordinator tells each service what to do next.
Choreography	Services react to events and publish events that trigger subsequent steps.

Messaging in a saga: styles



ถามไว้ให้คิด
คำถามคือ ทำไมต้องมี Saga pattern ที่ใช้แนวคิด separate local transaction
ใช้ distributed transaction ร่วมกับ 2PC (2-Phase Commit) แทนได้ไหม

Summary (1)



IPC is key

Microservices are a distributed architecture



Evolve APIs carefully

Manage how each service's API changes



Many technologies

Each with different trade-offs



Synchronous clients

Must be designed to handle partial failure

Summary (2)



Service discovery

Client side and service side



Messages and channels

A good model for messaging, typically broker-based

Q&A

Questions?

Interprocess communication in a microservice architecture

Further reading

Click a title to open it

- **Timeouts, retries and backoff with jitter**
  Amazon Builders' Library aws.amazon.com/builders-library/timeouts-retries…
- **Kafka: a Distributed Messaging System for Log Processing**
  Kreps et al., NetDB 2011 people.csail.mit.edu/matei/courses/2015/6.S897/r…
- **Resilience4j**
  Circuit breaker and retry library (successor to Hystrix) resilience4j.readme.io
- **gRPC documentation**
  Official documentation grpc.io/docs
- **OpenAPI Initiative**
  The REST API description standard www.openapis.org
- **Microservice patterns**
  Chris Richardson's pattern catalog microservices.io/patterns